Dr. RVR NRI INSTITUTE OF TECHNOLOGY

(An Institution Deemed to be University u/s 3 of UGC Act, 1956)
Pothavarappadu (V), Agiripalli (M), Eluru District, Andhra Pradesh - 521212

ACTIVITY-BASED LEARNING (ABL) ASSIGNMENT REPORT

PYTHON PROGRAMMING COURSE CASE STUDIES PORTFOLIO

Comprehensive Problem Decomposition, Algorithms, Flowcharts, Pseudocode, Python Implementations, Test Suites, and Analytical Findings for 10 Selected Problems

Student Name:	Student A
Roll Number:	[REDACTED]
Branch / Program:	B.Tech. - Computer Science & Engineering (CSE)
Section / Year:	Section A / I B.Tech. I Semester
Course Code & Title:	24CS101 - Python Programming
Assignment Category:	Activity-Based Learning (ABL - Dussehra Holidays Assignment)
Evaluation Weightage:	10 Marks (Evaluated by Faculty Member / Lab Mentor)
GitHub Repository:	Python-Activity-Based-Learning
Date of Submission:	October 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Dr. RVR NRI INSTITUTE OF TECHNOLOGY

BONA FIDE CERTIFICATE

This is to certify that this consolidated Activity-Based Learning (ABL) Assignment report entitled **"Python Programming - Activity-Based Learning: Portfolio of 10 Case Studies"** is a bona fide record of academic work successfully completed by **Student A** (Roll No: **[REDACTED]**), a student of **I B.Tech. in Computer Science & Engineering (Section A)**, at Dr. RVR NRI Institute of Technology, in partial fulfillment of the requirements for the Python Programming laboratory course during the academic year 2026–2027.

The student has independently selected 10 difficult-level case studies from the prescribed list of 100 problems, formulated structured computational algorithms, prepared flowcharts and pseudocode, implemented verified Python programs strictly adhering to the prescribed syllabus, executed multi-scenario test cases, and uploaded all verified materials to the dedicated GitHub repository **Python-Activity-Based-Learning**.

Course Coordinator Python Programming	**Lab Mentor / Evaluator** Dept. of CSE	**Dean / Head of Dept.** School of Computer Studies

STUDENT DECLARATION

I hereby declare that the Python solutions, algorithms, flowcharts, pseudocode, and test analyses presented in this report are my original work, developed strictly using the assigned Python syllabus concepts without the use of external non-syllabus libraries or pre-packaged packages. All outputs included herein have been verified through live terminal execution.

Date: October 2026 **Signature of Student:** Student A

TABLE OF CONTENTS & SYLLABUS MAPPING

#	Case Study ID & Title	Syllabus Concepts Applied	Evaluation Status
1	**12. Attendance Recovery Computation**	while loops, float arithmetic, tuple unpacking	Verified (100% Pass)
2	**17. Student Performance Trend Analysis**	Nested lists (3D matrix), difference loops, rule engine	Verified (100% Pass)
3	**28. Repeated Transaction Detection**	Pairwise nested loops, threshold comparisons, risk ledger	Verified (100% Pass)
4	**34. Student Feedback Sentiment Scoring**	String tokenization, character filtering, list matching	Verified (100% Pass)
5	**35. Text Compression Using Character Runs**	Run-length encoding (RLE), string indexing, lossless verification	Verified (100% Pass)
6	**42. Student Marks Matrix Analysis**	2D nested lists, row/column traversals, bubble sort ranks	Verified (100% Pass)
7	**45. Weekly Electricity Consumption Matrix**	2D grid matrix, appliance/day totals, slab tariff pricing	Verified (100% Pass)
8	**49. Tuple-Based Product Catalogue Audit**	Tuple immutability, data sanitization, asset valuation	Verified (100% Pass)
9	**53. Food Waste Monitoring System**	Lists of tuples, shrinkage math, financial loss audit	Verified (100% Pass)
10	**63. Multi-Product Price Fluctuation Analysis**	Chronological lists, volatility % formulas, risk sorting	Verified (100% Pass)

Prescribed Syllabus Compliance Guarantee:

As explicitly mandated by the course instructions, all 10 programs have been designed and implemented **using ONLY the prescribed foundational syllabus**: computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, nested lists, tuples, tuple operations, and unpacking. **No non-syllabus constructs** (such as dictionaries, sets, classes, or external modules like NumPy, Pandas, or SciPy) have been used in any of the core algorithms.

Case Study 12: Attendance Recovery Computation

1. Problem Statement

Calculates current attendance percentages and determines the minimum consecutive classes required to reach eligibility thresholds under different future attendance scenarios.

2. Objectives

• Compute floating-point attendance percentages from conducted and attended class counts.

• Simulate iterative recovery classes needed to reach the 75.0% statutory threshold.

• Model realistic future attendance scenarios (100%, 90%, 80% discipline).

• Calculate the maximum missable bunk buffer for already eligible students.

• Classify students into ELIGIBLE, CONDONATION, and CRITICAL SHORTAGE tiers.

3. Assumptions

• Attendance percentages are calculated with 2 decimal precision.

• Statutory university eligibility is set at 75.0%; condonation limit is 65.0%.

• Recovery simulation models the moving denominator effect (each new class increments both numerator and denominator).

• Input data is represented as a list of tuples (roll_no, student_name, total_classes, attended_classes).

4. Step-by-Step Algorithm

1. Read student records as list of tuples (roll_no, name, total_held, attended).

2. For each student, compute current_pct = (attended / total_held) * 100.0.

3. If current_pct >= 75.0%: Mark ELIGIBLE; compute bunk buffer while ((attended)/(total_held+1))*100 >= 75.0.

4. Else: If current_pct >= 65.0% mark CONDONATION, else CRITICAL; simulate while (sim_att/sim_tot)*100 < 75.0: sim_att += 1, sim_tot += 1.

5. Tabulate eligibility, recovery requirements across scenarios, and print cohort directives.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Read Tuples -> Calc current_pct -> Check >= 75%? -> If Yes: Status=ELIGIBLE, Calc Bunk Buffer -> If No: Check >= 65%? (CONDONATION/CRITICAL) -> Simulate Future Classes via while loop -> Print Audit Matrix -> End.

6. Formal Pseudocode

```
FUNCTION AttendanceRecovery(students, threshold=75.0):
    FOR EACH (roll, name, held, attended) IN students:
        pct = (attended / held) * 100
        IF pct >= threshold:
            status = "ELIGIBLE"
            rec_100 = 0
            bunk_buffer = CALC_BUFFER(attended, held, threshold)
        ELSE:
            status = "CONDONATION" IF pct >= 65.0 ELSE "CRITICAL"
            rec_100 = SIMULATE_RECOVERY(attended, held, threshold, rate=1.0)
        PRINT roll, name, pct, status, rec_100, bunk_buffer
    END FOR
```

7. Python Source Code Listing

```
Case Study 12: Attendance Recovery Computation
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Calculates current attendance percentages and determines the minimum consecutive
classes required to reach eligibility thresholds under different future attendance scenarios.
Concepts Used: Variables, Data Types, Operators, Conditional Statements, While/For Loops,
               Tuples, Tuple Unpacking, Lists, and String Formatting.
"""

def print_banner(title):
    border = "=" * 78
    print(border)
    print(title.center(78))
    print(border)

def calculate_attendance_percentage(attended, total):
    if total <= 0:
        return 0.0
    return round((attended / total) * 100.0, 2)

def calculate_consecutive_needed(attended, total, threshold, future_rate=1.0):
    """
    Simulates upcoming classes to reach target attendance percentage threshold.
    future_rate: proportion of upcoming classes attended (e.g., 1.0 for 100%, 0.8 for 80%).
    Returns the minimum number of total upcoming classes required.
    """
    current_pct = calculate_attendance_percentage(attended, total)
    if current_pct >= threshold:
        return 0

    # If future_rate is less than or equal to threshold / 100, student can never reach threshold
    if future_rate <= (threshold / 100.0):
        return -1  # Mathematically impossible to reach

    classes_needed = 0
    sim_attended = float(attended)
    sim_total = float(total)

    # Simulate step-by-step using while loop
    while (sim_attended / sim_total) * 100.0 < threshold:
        classes_needed += 1
        sim_total += 1
        sim_attended += future_rate

        # Guard against excessive simulation
        if classes_needed > 500:
            break

# ... [Full listing continues in attendance_recovery.py (160 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: Evaluated 8 students across Eligible (48/60, 52/60, 57/60), Condonation (42/60, 44/60), and Critical (32/60, 28/60) cohorts. All recovery simulations verified 100% mathematically accurate.

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

The moving denominator makes attendance recovery non-linear. The simulation algorithm successfully quantifies exact required classes without rounding errors, providing clear administrative guidance.

Case Study 17: Student Performance Trend Analysis

1. Problem Statement

Compares assessment scores across multiple examinations, calculates improvements and declines, identifies consistently weak subjects, and classifies students according to defined criteria.

2. Objectives

• Process multi-exam, multi-subject score matrices across chronological assessments.

• Calculate step-wise score differentials between Assessment 1, 2, and 3.

• Detect subjects where student marks are persistently below passing threshold (< 40).

• Classify student academic momentum into 5 distinct pedagogical categories.

• Generate curricular monitor reports highlighting systemic weakness.

3. Assumptions

• Assessments are scored out of 100; passing benchmark is 40.

• Assessments are chronological: Assessment 1 -> Assessment 2 -> Assessment 3.

• Student records are stored as (roll, name, 2D_scores_matrix).

• Matrix operations strictly use native list indexing and loops.

4. Step-by-Step Algorithm

1. Load cohort data containing 3 examination cycles across 4 curricular subjects.

2. For each student, compute individual exam averages and overall aggregate average.

3. Compute step-wise differentials and overall net trajectory (Exam 3 Avg - Exam 1 Avg).

4. For each subject, evaluate if marks are all < 40 ('Consistently Weak'), strictly increasing ('Improving'), strictly decreasing ('Declining'), or fluctuating.

5. Classify students: Outstanding Scholar (Avg >= 80, Shift >= 0), Consistent Improver (Shift >= +8), Remedial Support (Weak subject), Academic Warning (Shift <= -10).

6. Output student trajectory scorecards and cohort distribution summary.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Load 3D Matrix -> For Each Student -> Calculate Exam Averages -> For Each Subject -> Evaluate Score Trend (Weak/Improving/Declining/Fluctuating) -> Apply Classification Rules -> Print Scorecard -> Print Cohort Breakdown -> End.

6. Formal Pseudocode

```
FUNCTION AnalyzeTrends(cohort_data, subjects, exams):
    FOR EACH (roll, name, matrix) IN cohort_data:
        exam_avgs = [AVERAGE(matrix[e]) FOR e IN exams]
        net_shift = exam_avgs[LAST] - exam_avgs[FIRST]
        FOR EACH s IN subjects:
            subj_scores = [matrix[e][s] FOR e IN exams]
            trend = EVALUATE_TREND(subj_scores)
        category = CLASSIFY(overall_avg, net_shift, trends)
        PRINT student scorecard, trends, and category
    END FOR
```

7. Python Source Code Listing

```
Case Study 17: Student Performance Trend Analysis
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Compares assessment scores across multiple examinations, calculates improvements and declines,
identifies consistently weak subjects, and classifies students according to defined academic criteria.
Concepts Used: Nested Lists, Tuples, Tuple Unpacking, For/While Loops, Conditional Statements,
               String Manipulation, and List Accumulations.
"""

def print_banner(title):
    border = "=" * 80
    print(border)
    print(title.center(80))
    print(border)

def evaluate_subject_trend(scores, passing_mark=40):
    """
    Evaluates trend across assessments for a single subject:
    scores = [exam1, exam2, exam3]
    """
    n = len(scores)
    diffs = []
    for i in range(1, n):
        diffs.append(scores[i] - scores[i - 1])

    # Check consistently weak
    is_weak = True
    for s in scores:
        if s >= passing_mark:
            is_weak = False
            break

    if is_weak:
        trend = "Consistently Weak"
    elif all_positive(diffs):
        trend = "Improving (+)"
    elif all_negative(diffs):
        trend = "Declining (-)"
    else:
        trend = "Fluctuating"

    net_change = scores[-1] - scores[0]
    return trend, net_change

def all_positive(lst):
    for x in lst:
        if x <= 0:

# ... [Full listing continues in performance_trend.py (201 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: Tested across diverse academic archetypes: Improving (Student A +10.5, Charan +18.5), Declining (Aarav -12.5), Consistently Weak (Bhavya all < 40), and Steady (Divya +1.0). 100% correct classification.

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

Longitudinal trend analysis catches academic decay earlier than static semester exams. The rule-based engine successfully separated high performers from students needing urgent intervention.

Case Study 28: Repeated Transaction Detection & Fraud Audit

1. Problem Statement

Processes transaction identifiers and amounts in a list, detects duplicate transactions, identifies threshold violations, and generates a list of suspicious entries.

2. Objectives

• Ingest transaction records stored as tuples (tx_id, sender, receiver, amount, timestamp).

• Detect duplicated transaction identifiers (system replay or collision error).

• Identify duplicated transfer amounts between identical sender and receiver accounts.

• Flag single transactions exceeding the threshold limit (INR 50,000.00).

• Compute cumulative sender outflow using nested lists and flag velocity breaches (> INR 100,000.00).

3. Assumptions

• Amounts are positive real numbers.

• Data structures strictly avoid hash maps / dictionaries; nested list searching is used.

• Single threshold limit is INR 50,000.00; cumulative limit is INR 100,000.00.

4. Step-by-Step Algorithm

1. Ingest transaction records as list of tuples.

2. Check single limit violations: for each txn, if amount > 50000, flag 'HIGH' risk.

3. Perform pairwise comparison (i from 0 to N-1, j from i+1 to N-1): if id_i == id_j, flag 'CRITICAL' duplicate ID; if sender, receiver, and amount match, flag 'HIGH' duplicate transfer.

4. Aggregate cumulative volume per sender account into nested list [[sender, volume, count], ...].

5. If sender cumulative volume > 100000, flag transactions contributing to breach.

6. Output ingested ledger, suspicious entries table, and cumulative exposure matrix.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Load Txn Tuples -> Loop i: Check Amount > 50k -> Loop j: Check duplicate ID or duplicate transfer -> Aggregate Volume per Sender into List -> Check Volume > 100k -> Generate Forensic Audit Ledger -> End.

6. Formal Pseudocode

```
FUNCTION DetectSuspiciousTransactions(transactions):
    FOR i FROM 0 TO N-1:
        IF transactions[i].amount > 50000: FLAG_HIGH()
        FOR j FROM i+1 TO N-1:
            IF transactions[i].id == transactions[j].id: FLAG_CRITICAL_DUPLICATE_ID()
            ELSE IF SAME_TRANSFER(transactions[i], transactions[j]): FLAG_REPLAY_TRANSFER()
    ledger = AGGREGATE_SENDER_VOLUMES(transactions)
    FOR EACH s IN ledger:
        IF s.total > 100000: FLAG_VELOCITY_BREACH(s)
    PRINT audit tables
```

7. Python Source Code Listing

```
Case Study 28: Repeated Transaction Detection & Fraud Audit
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Processes transaction records stored in lists of tuples, detects duplicate transactions,
identifies threshold violations, and generates a structured audit log of suspicious entries.
Concepts Used: Lists, Tuples, Tuple Unpacking, Nested Loops, Conditional Statements,
               Arithmetic Computations, and Formatted Outputs.
"""

def print_banner(title):
    border = "=" * 85
    print(border)
    print(title.center(85))
    print(border)

def detect_suspicious_transactions(transactions, max_single_limit=50000.0, max_sender_limit=100000.0):
    """
    Scans transaction lists using pure list/tuple operations without sets or dicts.
    Identifies:
    1. Duplicate Transaction IDs.
    2. Duplicate Transfers (Identical Sender, Receiver, and Amount - Potential Replay).
    3. Single Transaction Threshold Violations (> max_single_limit).
    4. Cumulative Velocity Violations (Sender total exceeding max_sender_limit).
    """
    n = len(transactions)
    suspicious_records = []  # List of tuples: (tx_id, sender, receiver, amount, reason, risk_level)

    # 1. Check for Duplicate IDs & Duplicate Transfers via pairwise comparisons
    for i in range(n):
        id_i, sender_i, rec_i, amt_i, time_i = transactions[i]

        # Check single threshold violation
        if amt_i > max_single_limit:
            suspicious_records.append((id_i, sender_i, rec_i, amt_i, f"Exceeds Single Txn Limit (INR {max_single_limit:,.2f})",

        # Pairwise check for duplicate IDs or duplicate content
        for j in range(i + 1, n):
            id_j, sender_j, rec_j, amt_j, time_j = transactions[j]

            # Duplicate transaction ID
            if id_i == id_j:
                suspicious_records.append((id_j, sender_j, rec_j, amt_j, f"Duplicate Txn ID collision with index #{i+1}", "CRIT

            # Duplicate transfer (same sender, receiver, and amount)
            elif sender_i == sender_j and rec_i == rec_j and amt_i == amt_j:
                reason = f"Duplicate Transfer: Replay match with {id_i} (Amt: INR {amt_i:,.2f})"
                suspicious_records.append((id_j, sender_j, rec_j, amt_j, reason, "HIGH"))

# ... [Full listing continues in repeated_transactions.py (143 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: Successfully detected duplicate transfer (TX1002 and TX1004), single limit violation (TX1003 @ 65k), duplicate ID collision (TX1005 repeated), and cumulative breach (ACC_AMAR @ 125k).

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

The multi-pass list algorithm successfully flags high-value, duplicate, and rapid-succession fraud patterns without requiring external database or hashing engines.

Case Study 34: Student Feedback Sentiment Scoring

1. Problem Statement

Analyses multiple feedback sentences using a supplied keyword list, counts positive and negative indicators, calculates scores, and classifies feedback using predefined thresholds.

2. Objectives

• Sanitize raw feedback text by stripping punctuation and normalizing case.

• Tokenize sentences into word lists without third-party regex tools.

• Count domain-specific positive and negative keyword matches.

• Compute Net Sentiment Score (P - N) and Normalized Sentiment Index ((P - N) / (P + N)).

• Classify feedback into 6 descriptive categories and compute institutional approval ratings.

3. Assumptions

• Punctuation characters do not carry sentiment and are replaced with whitespace.

• Matched keywords within the domain lexicons have uniform weight.

• Feedback with zero matches is classified as NEUTRAL (No Keywords).

4. Step-by-Step Algorithm

1. Define positive and negative keyword lists.

2. Ingest feedback records as tuples (feedback_id, student_name, comment_string).

3. For each comment: iterate through characters, replace punctuation with space, convert to lowercase, split on whitespace.

4. Count positive and negative keyword hits using nested loops over keyword lists.

5. Compute Net Score = pos - neg; Norm Index = (pos - neg) / (pos + neg) if total > 0 else 0.0.

6. Classify: Highly Positive (Index >= 0.5), Positive (>0), Mixed (0.0 with keywords), Neutral (no keywords), Negative (>-0.5), Highly Negative (<=-0.5).

7. Calculate institutional approval percentage and identify top thematic keywords.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Load Lexicons & Feedback -> For Each Sentence -> Strip Punctuation -> Tokenize into Words -> Count Positive & Negative Matches -> Compute Net & Normalized Index -> Apply Thresholds -> Output Scorecards & Approval % -> End.

6. Formal Pseudocode

```
FUNCTION AnalyzeSentiment(feedbacks, pos_words, neg_words):
    FOR EACH (fid, author, text) IN feedbacks:
        tokens = CLEAN_AND_TOKENIZE(text)
        pos_cnt = COUNT_MATCHES(tokens, pos_words)
        neg_cnt = COUNT_MATCHES(tokens, neg_words)
        total = pos_cnt + neg_cnt
        norm_idx = (pos_cnt - neg_cnt) / total IF total > 0 ELSE 0.0
        category = CLASSIFY_INDEX(norm_idx, pos_cnt, neg_cnt)
        PRINT feedback scorecard with tokens, scores, and category
    END FOR
```

7. Python Source Code Listing

```
Case Study 34: Student Feedback Sentiment Scoring
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Analyses multiple student feedback sentences against supplied positive and negative
keyword lists, counts sentiment indicators, calculates normalized sentiment scores,
and classifies feedback according to predefined thresholds.
Concepts Used: Strings, String Cleaning & Tokenization, Lists, Nested Loops,
               Conditional Statements, Tuples, and Tabular Output Formatting.
"""

def print_banner(title):
    border = "=" * 80
    print(border)
    print(title.center(80))
    print(border)

def clean_and_tokenize(sentence):
    """
    Cleans punctuation and tokenizes sentence into words without external libraries.
    """
    cleaned_chars = []
    punctuation = ".,!?:;\"'()[]{}<>-/*_~"
    for ch in sentence:
        if ch in punctuation:
            cleaned_chars.append(" ")
        else:
            cleaned_chars.append(ch.lower())

    cleaned_string = "".join(cleaned_chars)
    tokens = cleaned_string.split()
    return tokens

def count_keyword_matches(tokens, keyword_list):
    """
    Counts matches of words in keyword_list using pure list traversal.
    Returns count and list of matched terms.
    """
    count = 0
    matched = []
    for token in tokens:
        for kw in keyword_list:
            if token == kw:
                count += 1
                matched.append(token)
                break
    return count, matched


# ... [Full listing continues in sentiment_scoring.py (199 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: Tested 7 qualitative statements: FB-01 (Highly Positive +1.0), FB-02 (Highly Negative -1.0), FB-03 (Positive +0.33), FB-06 (Neutral 0.0). Net course approval evaluated at 57.1%.

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

Normalized sentiment scoring removes sentence length bias, providing an automated and objective mechanism for academic administrators to evaluate student satisfaction.

Case Study 35: Text Compression Using Character Runs

1. Problem Statement

Encodes a string by grouping consecutive repeated characters and compares original and encoded lengths, including cases where encoding is not beneficial.

2. Objectives

• Implement Run-Length Encoding (RLE) to group consecutive identical characters.

• Build an inverse decoder to verify 100% lossless round-trip restoration.

• Compute compression space savings percentage: ((Orig - Enc) / Orig) * 100.

• Identify threshold cases where encoding causes payload expansion (negative savings).

• Formulate an automated decision engine: Compress vs Reject / Store Raw.

3. Assumptions

• Input strings contain printable ASCII characters.

• Character runs require identical adjacent characters.

• When encoded length >= original length, the engine recommends rejecting compression.

4. Step-by-Step Algorithm

1. Ingest input string text. If empty, return empty payload with 0% savings.

2. Scan string with while loop: count consecutive identical characters, append char + count to encoded parts.

3. Reconstruct original string from encoded text to verify lossless integrity.

4. Compute orig_len, comp_len, and savings_pct = ((orig_len - comp_len) / orig_len) * 100.

5. If comp_len < orig_len: Verdict = 'BENEFICIAL (Compress)'. Else: 'INEFFECTIVE (Reject / Store Raw)'.

6. Output comparative benchmarking ledger across test cases.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Read String -> While i < Length -> Count Consecutive Identical Chars -> Emit char + count -> Decompress Encoded String -> Verify Lossless -> Compute Savings % -> Encoded < Original? -> Yes: BENEFICIAL / No: INEFFECTIVE -> Output -> End.

6. Formal Pseudocode

```
FUNCTION EncodeRLE(text):
    IF LENGTH(text) == 0: RETURN "", 0, 0, 0.0, "EMPTY", TRUE
    encoded = []
    i = 0
    WHILE i < LENGTH(text):
        ch = text[i]; count = 1
        WHILE i + 1 < LENGTH(text) AND text[i+1] == ch:
            count += 1; i += 1
        encoded.APPEND(ch + STR(count))
        i += 1
    enc_str = JOIN(encoded)
    savings = ((LENGTH(text) - LENGTH(enc_str)) / LENGTH(text)) * 100
    verdict = "BENEFICIAL" IF LENGTH(enc_str) < LENGTH(text) ELSE "INEFFECTIVE"
    RETURN enc_str, savings, verdict, (DECODE(enc_str) == text)
```

7. Python Source Code Listing

```
Case Study 35: Text Compression Using Character Runs (Run-Length Encoding)
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Encodes input strings by grouping consecutive repeated characters into (char + run_count),
evaluates lossless compressibility, and identifies threshold cases where encoding produces
negative savings or expansions.
Concepts Used: Strings, String Indexing & Slicing, While/For Loops, Conditionals,
               Tuples, Lists, and Verification Algorithms.
"""

def print_banner(title):
    border = "=" * 80
    print(border)
    print(title.center(80))
    print(border)

def encode_character_runs(text):
    """
    Performs Run-Length Encoding on an input string.
    Returns: encoded_string, list_of_runs as [(char, count), ...]
    """
    if len(text) == 0:
        return "", []

    encoded_parts = []
    runs = []

    n = len(text)
    i = 0
    while i < n:
        curr_char = text[i]
        run_len = 1

        while i + 1 < n and text[i + 1] == curr_char:
            run_len += 1
            i += 1

        runs.append((curr_char, run_len))
        encoded_parts.append(curr_char + str(run_len))
        i += 1

    encoded_text = "".join(encoded_parts)
    return encoded_text, runs

def decode_character_runs(encoded_text):
    """
    Decompresses run-length encoded text to verify lossless integrity.

# ... [Full listing continues in text_compression.py (154 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: Tested 9 strings: Highly repetitive (+67.5% savings), Genomic (+67.7%), Non-repeating (-100% expansion), Alternating (-100% expansion), Single char (+85.0%). 100% lossless restoration verified.

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

RLE requires an average run length > 2 to yield savings. For non-repeating data, appending integer counts doubles payload size, proving why compression engines must inspect entropy before encoding.

Case Study 42: Student Marks Matrix Analysis

1. Problem Statement

Analyses a matrix of student marks across subjects, calculates row-wise and column-wise statistics, identifies top performers, and detects subjects with low average performance.

2. Objectives

• Construct and process a 2D marks matrix (N students x M subjects).

• Execute row-wise traversals to compute student total marks, percentages, and grades.

• Implement in-place sorting to assign ranks in descending order of merit.

• Execute column-wise traversals to compute subject class averages, maximums, and pass rates.

• Detect curricular subjects with concerning low average performance (< 65.0%).

3. Assumptions

• All subjects have max score of 100; passing threshold is 40.

• Quality benchmark for subject class average is set at 65.0%.

• Strictly native 2D list manipulation without NumPy or Pandas.

4. Step-by-Step Algorithm

1. Load 2D marks matrix, student names, and subject list.

2. Row-Wise Analysis: compute row sum, percentage, and letter grade (O, A+, A, B, C, F).

3. Sort student records by percentage descending using bubble sort and assign ranks 1 to N.

4. Column-Wise Analysis: extract column marks, compute class average, max, min, and pass rate (mark >= 40).

5. If subject class average < 65.0%: flag as 'LOW PERFORMANCE [ALERT]'.

6. Output ranked student scorecard matrix, subject diagnostics, merit podium, and remedial alerts.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Load 2D Matrix -> Row Loop: Compute Student Total, Pct, Grade -> Sort Records & Assign Ranks -> Column Loop: Compute Subject Avg, Max, Min, Pass Rate -> Check Avg < 65%? -> Flag Alerts -> Print Scorecards & Podium -> End.

6. Formal Pseudocode

```
FUNCTION AnalyzeMarksMatrix(students, subjects, matrix, benchmark=65.0):
    // Row-wise
    FOR r FROM 0 TO N-1:
        tot = SUM(matrix[r]); pct = tot / (M * 100) * 100
        records.APPEND([students[r], tot, pct, GET_GRADE(pct)])
    SORT_BY_PERCENTAGE_DESCENDING(records)
    // Column-wise
    FOR c FROM 0 TO M-1:
        col_marks = [matrix[r][c] FOR r IN 0..N-1]
        avg = AVERAGE(col_marks)
        IF avg < benchmark: FLAG_LOW_SUBJECT(subjects[c], avg)
    PRINT scorecards, diagnostics, and podium
```

7. Python Source Code Listing

```
Case Study 42: Student Marks Matrix Analysis
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Analyses a 2D matrix of student examination marks across multiple academic subjects.
Computes row-wise student metrics (totals, percentages, grades, ranks), column-wise
subject diagnostics (averages, highest/lowest marks, pass percentages), and detects
subjects with concerning low average performance.
Concepts Used: 2D Nested Lists, Row-wise & Column-wise Traversals, Nested Loops,
               Conditionals, Tuples, Sorting via Algorithmic Swapping, and Formatted Grids.
"""

def print_banner(title):
    border = "=" * 88
    print(border)
    print(title.center(88))
    print(border)

def determine_grade(percentage):
    if percentage >= 90.0:
        return "O  (Outstanding)"
    elif percentage >= 80.0:
        return "A+ (Excellent)"
    elif percentage >= 70.0:
        return "A  (Very Good)"
    elif percentage >= 60.0:
        return "B  (Good)"
    elif percentage >= 50.0:
        return "C  (Average)"
    else:
        return "F  (Fail)"

def analyze_marks_matrix(students, subjects, marks_matrix, benchmark_avg=65.0):
    num_students = len(students)
    num_subjects = len(subjects)

    print_banner("ACADEMIC PERFORMANCE MATRIX EVALUATION")
    print(f"Cohort Size: {num_students} Students | Curricular Subjects: {num_subjects}")
    print(f"Institutional Quality Benchmark Subject Average: {benchmark_avg}%\n")

    # 1. Row-Wise Analysis (Student Statistics)
    # student_results: [ [student_name, [scores], total, percentage, grade, rank], ... ]
    student_records = []

    for r in range(num_students):
        scores = marks_matrix[r]
        row_total = sum(scores)
        pct = round(row_total / (num_subjects * 100.0) * 100.0, 2)

# ... [Full listing continues in marks_matrix.py (163 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: Analyzed 7 students across 5 subjects. Student A ranked #1 (91.4% [O]), Farhan #2 (83.6% [A+]), Bhavya #3 (81.6% [A+]). Detected Discrete Math (avg 56.4%, pass rate 71.4%) as below benchmark.

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

Orthogonal 2D list traversal efficiently delivers both micro-level student evaluation and macro-level departmental insights, identifying subject-level academic distress for targeted faculty tutorials.

Case Study 45: Weekly Electricity Consumption Matrix

1. Problem Statement

Processes appliance-wise electricity readings across seven days, calculates daily and appliance totals, and identifies consumption exceeding specified limits.

2. Objectives

• Model appliance electricity readings across a 7-day week using 2D nested lists.

• Compute weekly totals and daily averages per appliance (row-wise).

• Compute daily cumulative household consumption across all appliances (column-wise).

• Detect peak single consumption events across all appliances and days.

• Flag daily safety breaches (> 24.0 kWh/day) and appliance quota breaches (> 35.0 kWh/week).

• Calculate weekly power bills using tiered multi-slab utility tariffs.

3. Assumptions

• Readings are recorded in kWh across Monday through Sunday.

• Daily safety limit = 24.0 kWh; weekly appliance quota = 35.0 kWh.

• Tariff slabs: First 50 @ INR 3.50, next 50 @ INR 5.00, >100 @ INR 7.50; fixed charge = INR 65.00.

4. Step-by-Step Algorithm

1. Load 6 appliances x 7 days energy matrix.

2. Row Aggregation: compute weekly sum and daily average per appliance; flag if total > 35.0 kWh; scan for peak single reading.

3. Column Aggregation: compute daily total across appliances; flag days where total > 24.0 kWh.

4. Compute grand total consumption = sum of daily totals.

5. Calculate tiered electricity charges and add fixed grid maintenance fee.

6. Output consumption grid, peak event, threshold violations, and billing breakdown.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Load 2D Energy Matrix -> Row Loop: Appliance Totals, Daily Avg, Quota Checks, Peak Update -> Column Loop: Daily Household Totals, Safety Checks -> Compute Slab Tariff -> Print Consumption Grid & Bill -> End.

6. Formal Pseudocode

```
FUNCTION AnalyzeElectricity(appliances, days, matrix, daily_lim=24.0, app_lim=35.0):
    FOR r FROM 0 TO R-1:
        r_tot = SUM(matrix[r])
        IF r_tot > app_lim: FLAG_APPLIANCE_QUOTA(appliances[r])
        UPDATE_PEAK_EVENT(matrix[r])
    FOR c FROM 0 TO C-1:
        d_tot = SUM_COLUMN(matrix, c)
        IF d_tot > daily_lim: FLAG_DAILY_LIMIT(days[c])
    bill = COMPUTE_SLAB_TARIFF(SUM(d_tot))
    PRINT grid, surge analysis, alerts, and bill
```

7. Python Source Code Listing

```
Case Study 45: Weekly Electricity Consumption Matrix
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Processes appliance-wise electrical consumption readings across 7 days of the week.
Computes daily and appliance-wise totals, identifies quota threshold violations,
detects peak consumption surges, and estimates weekly billing costs.
Concepts Used: 2D Nested Lists, Row & Column Iterations, Min/Max Detection,
               Conditional Statements, Tuples, and Tabular Formatting.
"""

def print_banner(title):
    border = "=" * 88
    print(border)
    print(title.center(88))
    print(border)

def calculate_electricity_bill(total_units):
    """
    Tiered slab tariff:
    - First 50 units  @ INR 3.50 / unit
    - Next 50 units   @ INR 5.00 / unit
    - Above 100 units @ INR 7.50 / unit
    Fixed charges     : INR 65.00
    """
    fixed_charge = 65.0
    energy_charge = 0.0
    remaining = total_units

    if remaining > 100:
        energy_charge += (remaining - 100) * 7.50
        remaining = 100
    if remaining > 50:
        energy_charge += (remaining - 50) * 5.00
        remaining = 50
    energy_charge += remaining * 3.50

    total_bill = energy_charge + fixed_charge
    return round(energy_charge, 2), fixed_charge, round(total_bill, 2)

def analyze_electricity_matrix(appliances, days, consumption_matrix, daily_limit=24.0, appliance_limit=35.0):
    num_appliances = len(appliances)
    num_days = len(days)

    print_banner("HOUSEHOLD WEEKLY ELECTRICITY CONSUMPTION AUDIT")
    print(f"Audit Horizon           : 7 Full Operating Days ({days[0]} to {days[-1]})")
    print(f"Daily Safety Limit      : {daily_limit:.1f} kWh / day")
    print(f"Weekly Appliance Quota  : {appliance_limit:.1f} kWh / week\n")

# ... [Full listing continues in electricity_consumption.py (164 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: Weekly total = 155.30 kWh. Peak event: 12.0 kWh by Air Conditioner on Sunday. AC exceeded quota (64.0 kWh vs 35.0 limit). Weekend breaches: Sat (27.5 kWh) and Sun (28.4 kWh). Total Bill: INR 904.75.

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

Appliance-level matrix auditing accurately isolates excessive power consumers. The AC alone contributed 41.2% of consumption, proving that targeted conservation yields immediate financial savings.

Case Study 49: Tuple-Based Product Catalogue Audit

1. Problem Statement

Stores product identifiers, prices, and stock quantities in tuples; detects invalid records, calculates inventory values, and identifies items requiring replenishment.

2. Objectives

• Store product catalogue records in immutable tuples (id, name, category, price, stock, reorder).

• Quarantine corrupt or invalid tuples (negative prices, negative stock, blank names).

• Compute item-wise and category-wise inventory asset valuations.

• Identify items breaching reorder thresholds (stock <= reorder level).

• Compute optimal purchase order quantities and required procurement capital.

3. Assumptions

• Tuples strictly contain 6 elements in defined order.

• Valid prices must be > 0; stock and reorder thresholds must be >= 0.

• Reorder target buffer is max(2 * reorder, reorder + 10).

4. Step-by-Step Algorithm

1. Ingest raw catalogue records as list of tuples.

2. Validation: check tuple length, non-empty ID and name, positive price, non-negative stock and reorder. Separate valid from quarantined invalid records.

3. Valuation: for each valid tuple, unpack (pid, name, cat, price, stock, reorder); compute item_val = price * stock; accumulate total valuation and units.

4. Aggregate valuations by category into nested list [[cat, val, units], ...].

5. Replenishment: if stock <= reorder, compute order_qty = target - stock, estimated_cost = order_qty * price; flag CRITICAL if stock == 0.

6. Output quarantined error log, inventory valuation ledger, category shares, and purchase orders.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Load Tuples -> For Each Tuple -> Validate Schema & Values -> Invalid? Quarantine Record -> Valid? Unpack Tuple -> Calc Valuation -> Aggregate Category -> Stock <= Reorder? -> Yes: Generate Purchase Order -> Print Reports -> End.

6. Formal Pseudocode

```
FUNCTION AuditCatalogue(raw_tuples):
    FOR EACH t IN raw_tuples:
        IF VALID(t): valid_list.APPEND(t)
        ELSE: invalid_list.APPEND(t)
    FOR EACH (pid, name, cat, price, stock, reorder) IN valid_list:
        val = price * stock
        grand_total += val
        IF stock <= reorder:
            order_qty = MAX(2*reorder, reorder+10) - stock
            orders.APPEND((pid, name, order_qty, order_qty * price))
    PRINT quarantined, inventory ledger, categories, and purchase orders
```

7. Python Source Code Listing

```
Case Study 49: Tuple-Based Product Catalogue Audit
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Maintains product catalogue records stored in immutable tuples.
Validates records against integrity rules, calculates granular inventory valuations,
identifies inventory requiring replenishment, and computes reorder capital requirements.
Concepts Used: Tuples, Tuple Immutability & Packing/Unpacking, Lists of Tuples,
               Conditionals, Nested Loops, Error Segregation, and Formatted Ledgers.
"""

def print_banner(title):
    border = "=" * 88
    print(border)
    print(title.center(88))
    print(border)

def validate_catalogue_record(record):
    """
    Validates tuple structure:
    (product_id, product_name, category, unit_price, stock_quantity, reorder_threshold)
    Returns: (is_valid, error_reason)
    """
    if len(record) != 6:
        return False, f"Malformed tuple length ({len(record)} elements, expected 6)"

    pid, name, cat, price, stock, reorder = record

    if not isinstance(pid, str) or len(pid.strip()) == 0:
        return False, "Invalid or missing Product ID"

    if not isinstance(name, str) or len(name.strip()) == 0:
        return False, "Invalid or missing Product Name"

    if not isinstance(price, (int, float)) or price <= 0:
        return False, f"Invalid unit price ({price}): Must be positive numeric"

    if not isinstance(stock, int) or stock < 0:
        return False, f"Invalid stock quantity ({stock}): Must be non-negative integer"

    if not isinstance(reorder, int) or reorder < 0:
        return False, f"Invalid reorder threshold ({reorder}): Must be non-negative integer"

    return True, "Valid"

def audit_product_catalogue(raw_catalogue):
    print_banner("ENTERPRISE PRODUCT CATALOGUE INTEGRITY & INVENTORY AUDIT")
    print(f"Total Catalogue Records Ingested: {len(raw_catalogue)}\n")

# ... [Full listing continues in product_catalogue.py (175 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: 11 records ingested. Quarantined 3 invalid tuples (negative price, blank name, negative stock). Evaluated 8 valid items: Warehouse Asset Value = INR 358,634.00. Generated 4 purchase orders (Req Capital: INR 662,966.00).

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

Tuple immutability protects master data integrity. The dual-phase audit cleanly quarantines corrupt records and automates inventory replenishment planning.

Case Study 53: Food Waste Monitoring System

1. Problem Statement

Compares prepared meals, meals served, and leftovers across several days; calculates wastage percentages and identifies days exceeding acceptable limits.

2. Objectives

• Model dining hall catering logs as tuples: (day, prepared, served, leftovers).

• Compute unaccounted shrinkage: Prepared - (Served + Leftovers).

• Calculate daily and cumulative percentage food waste.

• Flag days breaching the acceptable sustainability limit (> 10.0%).

• Quantify financial losses at standard meal production cost (INR 65.00/meal).

• Provide actionable operational recommendations to minimize institutional waste.

3. Assumptions

• Quantities are non-negative integers.

• Discrepancy != 0 indicates unrecorded theft, kitchen spillage, or counting errors.

• Standard meal unit cost is INR 65.00; acceptable waste ceiling is 10.0%.

4. Step-by-Step Algorithm

1. Load daily dining hall records as list of tuples.

2. For each day, unpack (day, prep, served, leftovers).

3. Compute discrepancy = prep - (served + leftovers); waste_pct = (leftovers / prep) * 100; loss = leftovers * 65.00.

4. Update cumulative totals (prepared, served, leftovers, financial loss).

5. Evaluate status: if waste_pct > 10.0%: 'EXCESSIVE [BREACH]'; else if discrepancy != 0: 'DISCREPANCY ALERT'; else: 'ACCEPTABLE [OK]'.

6. Identify worst wastage day and most efficient day.

7. Output audit ledger, sustainability metrics, breach directives, and recommendations.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Load Daily Logs -> For Each Day -> Unpack Tuple -> Calc Discrepancy & Waste % -> Calc Financial Loss -> Waste % > 10%? -> Flag EXCESSIVE BREACH -> Discrepancy != 0? -> Flag DISCREPANCY ALERT -> Print Audit Matrix & Recommendations -> End.

6. Formal Pseudocode

```
FUNCTION MonitorFoodWaste(records, threshold=10.0, cost=65.0):
    FOR EACH (day, prep, served, left) IN records:
        disc = prep - (served + left)
        pct = (left / prep) * 100
        loss = left * cost
        IF pct > threshold: FLAG_BREACH(day, pct, loss)
        ELSE IF disc != 0: FLAG_DISCREPANCY(day, disc)
        UPDATE_CUMULATIVE_METRICS(prep, served, left, loss)
    PRINT ledger, metrics, breach directives, and recommendations
```

7. Python Source Code Listing

```
Case Study 53: Food Waste Monitoring System
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Audits institutional dining hall food utilization across operating days.
Compares prepared meals, actual meals served, and leftover quantities.
Calculates percentage wastage, flags days breaching acceptable waste thresholds,
detects unaccounted inventory shrinkage, and computes financial wastage costs.
Concepts Used: Lists of Tuples, Tuple Unpacking, Arithmetic Operations, Loops,
               Conditional Statements, and Executive Tabular Reporting.
"""

def print_banner(title):
    border = "=" * 88
    print(border)
    print(title.center(88))
    print(border)

def analyze_food_waste(daily_records, acceptable_threshold=10.0, meal_unit_cost=65.0):
    num_days = len(daily_records)

    print_banner("INSTITUTIONAL DINING HALL FOOD WASTE & SUSTAINABILITY AUDIT")
    print(f"Audit Horizon               : {num_days} Operating Days")
    print(f"Acceptable Wastage Limit    : {acceptable_threshold:.1f}% of Prepared Volume")
    print(f"Standard Meal Production Cost: INR {meal_unit_cost:,.2f} / plate\n")

    row_fmt = "{:<12} {:>10} {:>10} {:>10} {:>12} {:>10} {:>14} {:<18}"
    headers = ("Day", "Prepared", "Served", "Leftover", "Discrepancy", "Waste %", "Loss (INR)", "Audit Status")
    print(row_fmt.format(*headers))
    print("-" * 102)

    total_prepared = 0
    total_served = 0
    total_leftovers = 0
    total_discrepancy = 0
    total_financial_loss = 0.0

    breach_days = []

    max_waste_pct = -1.0
    max_waste_day = ""
    min_waste_pct = 999.0
    min_waste_day = ""

    for record in daily_records:
        day_name, prep, served, leftovers = record

        # Unaccounted meals (shrinkage/spillage)

# ... [Full listing continues in food_waste_monitoring.py (136 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: Monitored 8 days (6,150 meals prepared, 5,495 served, 630 wasted). Overall waste = 10.24% (Financial Loss: INR 40,950.00). Flagged Friday (16.84%) and Sunday (17.33%) as major breach days. Detected 25 unaccounted meals.

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

Wastage spikes systematically on weekends due to student travel. Mathematical monitoring enables dynamic kitchen batch resizing and flags inventory leakage.

Case Study 63: Multi-Product Price Fluctuation Analysis

1. Problem Statement

Processes historical product prices stored in lists, calculates percentage changes, identifies maximum fluctuations, and flags products exceeding a supplied threshold.

2. Objectives

• Model multi-period commodity price histories using lists of floating-point values.

• Calculate period-over-period percentage price shifts: ((P_t - P_t-1) / P_t-1) * 100.

• Compute peak-to-trough range volatility: ((Max - Min) / Min) * 100.

• Detect maximum single-period price fluctuation for each commodity.

• Flag high-risk volatile commodities breaching the safety threshold (> 15.0%).

• Sort and rank portfolio commodities from most volatile to most stable.

3. Assumptions

• Price observations are chronologically ordered at uniform monthly intervals.

• Prices are strictly positive real numbers.

• Volatility alert threshold is set at 15.0% maximum single-period swing.

4. Step-by-Step Algorithm

1. Load commodity records: list of tuples (id, name, category, price_list).

2. For each commodity: iterate through price_list from index 1 to T-1; compute shift = ((P[t] - P[t-1]) / P[t-1]) * 100; track max_abs_shift.

3. Compute min_p, max_p, peak-to-trough volatility = ((max - min) / min) * 100, and net overall change.

4. If max_abs_shift > 15.0%: mark 'VOLATILE [FLAGGED]'; else if > 7.5%: 'MODERATE'; else: 'HIGHLY STABLE'.

5. Sort audit results descending by max_abs_shift using in-place bubble sort.

6. Output historical price matrix, volatility audit ledger, risk rankings, and hedging directives.

5. Flowchart Architectural Representation

Flowchart Process Description: Start -> Load Historical Prices -> For Each Commodity -> Compute Period Shifts -> Find Max Shift, Min, Max & Range Volatility -> Max Shift > 15%? -> Flag VOLATILE -> Sort Products by Volatility -> Print Matrix & Hedging Alerts -> End.

6. Formal Pseudocode

```
FUNCTION AnalyzePriceFluctuations(commodities, threshold=15.0):
    FOR EACH (id, name, cat, prices) IN commodities:
        shifts = [((prices[t]-prices[t-1])/prices[t-1])*100 FOR t IN 1..T-1]
        max_fluct = MAX(ABS(shifts))
        range_vol = ((MAX(prices) - MIN(prices)) / MIN(prices)) * 100
        status = "VOLATILE" IF max_fluct > threshold ELSE "STABLE"
        records.APPEND([id, name, max_fluct, range_vol, status])
    SORT_BY_VOLATILITY_DESCENDING(records)
    PRINT prices, volatility ledger, risk ranks, and directives
```

7. Python Source Code Listing

```
Case Study 63: Multi-Product Price Fluctuation Analysis
Department of Computer Science & Engineering
Dr. RVR NRI Institute of Technology

Description:
Tracks historical price trajectories of multiple retail and wholesale commodities over time.
Computes period-over-period percentage changes, peak-to-trough price variance,
identifies maximum single-period price fluctuations, and flags volatile products
breaching acceptable risk thresholds.
Concepts Used: Lists of Prices, Nested Loops, Min/Max Algorithms, Arithmetic Formulas,
               Conditional Statements, Tuples, Sorting via Algorithmic Swapping, and Tabular Reporting.
"""

def print_banner(title):
    border = "=" * 88
    print(border)
    print(title.center(88))
    print(border)

def calculate_fluctuations(price_series):
    """
    Computes period-over-period percentage shifts, maximum absolute shift,
    peak-to-trough range, and net change.
    """
    n = len(price_series)
    if n < 2:
        return [], 0.0, 0.0, 0.0

    shifts = []
    max_abs_shift = 0.0

    for i in range(1, n):
        prev = price_series[i - 1]
        curr = price_series[i]
        pct = round(((curr - prev) / prev) * 100.0, 2)
        shifts.append(pct)
        if abs(pct) > max_abs_shift:
            max_abs_shift = abs(pct)

    min_p = min(price_series)
    max_p = max(price_series)
    peak_trough_volatility = round(((max_p - min_p) / min_p) * 100.0, 2) if min_p > 0 else 0.0
    net_change = round(((price_series[-1] - price_series[0]) / price_series[0]) * 100.0, 2)

    return shifts, max_abs_shift, peak_trough_volatility, net_change

def analyze_market_fluctuations(products_dataset, periods, threshold_pct=15.0):
    num_products = len(products_dataset)
    num_periods = len(periods)

# ... [Full listing continues in price_fluctuation.py (142 total lines)]
```

8. Test Cases & Verification Summary

Verification Finding: Tracked 6 commodities across 6 months (Jan-Jun). Tomatoes (+71.88% shift, 240% range vol), Onions (+42.86% shift, 133% range vol), and Sunflower Oil (+18.31% shift) flagged volatile. Rice, Sugar, and Wheat verified stable.

9. Verified Execution Terminal Output



10. Conclusion & Analytical Insights

Agricultural perishables exhibit extreme price volatility compared to staple grains. Algorithmic volatility tracking enables procurement teams to deploy dynamic hedging and renegotiate fixed supply contracts.